UVM Interview Questions:

1. What is *uvm_transaction*, *uvm_seq_item, uvm_object, uvm_component*?

**Ans:** uvm_object: Core class based operational methods (create, copy, clone, compare, print, record, etc..), instance identification fields (name, type name, unique id, etc.) and random seeding were defined in it
uvm_component: Components are quasi-static objects that exist throughout the simulation.
uvm_transaction: Used in stimulus generation and analysis.
Uvm Sequence Item: The sequence-item defines the pin level activity generated by agent (to drive to DUT through the driver) or the activity has to be observed by agent (Placeholder for the activity monitored by the monitor on DUT signals).

2. What is the advantage of `uvm_component_utils() and `uvm_object_utils() ?

Ans: The reason there are two macros is because the factory design pattern fixes the number of arguments that a constructor can have. Classes derived from uvm_object have constructors with one argument, a string name. Classes derived from uvm_component have two arguments, a name and a uvm_component parent.

3. What is the difference between `uvm_do and `uvm_ran_send?

Ans: `uvm_do perform the below steps:

1. create
2. start_item
3. randomize
4. finish_item
5. get_response (optional)

while `uvm_rand_send perform all the above steps except create. User needs to create sequence / sequence_item.

4. diff between *uvm_transaction* and *uvm_seq_item*?

Ans: class uvm_sequence_item extends uvm_transaction

uvm_sequence_item extended from uvm_transaction only, uvm_sequence_item class has more functionality to support sequence & sequencer features. uvm_sequence_item provides the hooks for sequencer and sequence , So you can generate transaction by using sequence and sequencer , and uvm_transaction provide only basic methods like do_print and do_record etc .

5. What is the difference between uvm _virtual_sequencer and uvm_sequencer?
   Ans:
6. What are the benefits of using UVM?
   Ans: Some of the benefits of using UVM are :

   **Modularity and Reusability** – The methodology is designed as modular components (Driver, Sequencer, Agents , env etc) which enables reusing components across unit level to multi-unit or chip level verification as well as across projects.
   **Separating Tests from Testbenches** – Tests in terms of stimulus/sequencers are kept separate from the actual testbench hierarchy and hence there can be reuse of stimulus across different units or across projects
   **Simulator independent** – The base class library and the methodology is supported by all simulators and hence there is no dependence on any specific simulator
   **Better control on Stimulus generation** – Sequence methodology gives good control on stimulus generation. There are several ways in which sequences can be developed which includes randomization, layered sequences, virtual sequences etc which provides a good control and rich stimulus generation capability.
   **Easy configuration** – Config mechanisms simplify configuration of objects with deep hierarchy. The configuration mechanism helps in easily configuring different testbench components based on which verification environment uses it and without worrying about how deep any component is in testbench hierarchy
   **Factory mechanism** – Factory mechanisms simplifies modification of components easily. Creating each components using factory enables them to be overridden in different tests or environments without changing underlying code base.

7. What is the super keyword? What is the need of calling super.build() and super.connect()?
   Ans:
8. Is uvm is independent of systemverilog ?
   Ans: UVM is a methodology based on SystemVerilog language and is not a language on its own. It is a standardized methodology that defines several best practices in verification to enable efficiency in terms of reuse and is also currently part of IEEE 1800.2 working group.
9. Can we have a user-defined phase in UVM?

Ans: In addition to the predefined phases available in uvm, the user has the option to add his own phase to a component. This is typically done by extending the uvm_phase class the constructor needs to call super.new which has three arguments

Name of the phase task or function
Top down or bottom up phase
Task or function

The call_task or call_func and get_type_name need to be implemented to complete the addition of new phase. Below is a simple example

**Example**

```
class custom_phase extends uvm_phase;
  function new();
    super.new("custom",1,1);
  endfunction

  task call_task  ( uvm_component parent);
    my_comp_type comp;
    if ( $cast(comp,parent) )
         comp.custom_phase();
  endtask

  virtual function string get_type_name();
    return "custom";
  endfunction
endclass
```

10. What is p_sequencer?

Ans: **p_sequencer** is a typed-specific sequencer pointer, created by registering the sequence to the sequencer using macros (`uvm_declare_p_sequencer) . Being type specific, you will be able to access anything added to the sequencer (i.e. pointers to other sequencers, etc.). p_sequencer will not exist if we have not registered the sequence with the `uvm_declare_p_sequencer macros.

11. What is the uvm RAL model? why it is required?

Ans: In a verification context, a register model (or register abstraction layer) is a set of classes that model the memory mapped behavior of registers and memories in the DUT in order to facilitate stimulus generation and functional checking (and optionally some aspects of functional coverage). The UVM provides a set of base classes that can be extended to implement comprehensive register modeling capabilities.

12. What is the difference between new() and create?

Ans: We all know about new() method that is use to allocate memory to an object instance. In UVM (and OVM), the create() method causes an object instance to be created from the factory. This allows you to use factory overrides to replace the desired object with an object of a different type without having to recode.

13. What is an analysis port?

Ans: Analysis port (class uvm_tlm_analysis_port) — a specific type of transaction-level port that can be connected to zero, one, or many analysis exports and through which a component may call the method write implemented in another component, specifically a subscriber.
port, export, and imp classes used for transaction analysis.

**uvm_analysis_port**
Broadcasts a value to all subscribers implementing a uvm_analysis_imp.
**uvm_analysis_imp**
Receives all transactions broadcasted by a uvm_analysis_port.
**uvm_analysis_export**
Exports a lower-level uvm_analysis_imp to its parent.

14. What is TLM FIFO?

Ans: In simpler words TLM FIFO is a FIFO between two UVM components, preferably between Monitor and Scoreboard. Monitor keep on sending the DATA, which will be stored in TLM FIFO, and Scoreboard can get data from TLM FIFO whenever needed.

```
// Create a FIFO with depth 4
     tlm_fifo = new ("uvm_tlm_fifo", this, 4);
```

15. How the sequence starts?

Ans: start_item starts the sequence

```
virtual task start_item ( uvm_sequence_item  item,
                          int  set_priority  =  -1,
                         uvm_sequencer_base  sequencer  =  null )
```

start_item and finish_item together will initiate operation of a sequence item. If the item has not already been initialized using create_item, then it will be initialized here to use the default sequencer specified by m_sequencer.

16. What is the difference between UVM RAL model backdoor write/read and front door write/read?

Ans: Fontdoor access means using the standard access mechanism external to the DUT to read or write to a register. This usually involves sequences of time-consuming transactions on a bus interface.

Backdoor access means accessing a register directly via hierarchical reference or outside the language via the PLI. A backdoor reference usually in 0 simulation time.

17. What is an objection?

Ans: The objection mechanism in UVM is to allow hierarchical status communication among components which is helpful in deciding the end of test.

There is a built-in objection for each in-built phase, which provides a way for components and objects to synchronize their testing activity and indicate when it is safe to end the phase and, ultimately, the test end.

The component or sequence will raise a phase objection at the beginning of an activity that must be completed before the phase stops, so the objection will be dropped at the end of that activity. Once all of the raised objections are dropped, the phase terminates.

Raising an objection: phase.raise_objection(this);

Dropping an objection: phase.drop_objection(this);

18. What is the advantage of `uvm_pre_body and `uvm_post_body?

Ans:

19. What is the difference between Active mode and Passive mode?

Ans: An agent is a collection of a sequencer, a driver and a monitor.

In **active mode**, the sequencer and the driver are constructed and stimulus is generated by sequences sending sequence items to the driver through the sequencer. At the same time the monitor assembles pin level activity into analysis transactions.

In **passive mode**, only the monitor is constructed and it performs the same function as in an active agent. Therefore, your passive agent has no need for a sequencer. You can set up the monitor using a configuration object.

20. What is the difference between copy and clone?

Ans: The built-in **copy()** method executes the __m_uvm_field_automation() method with the required copy code as defined by the field macros (if used) and then calls the built-in **do_copy()** virtual function. The built-in do_copy() virtual function, as defined in the uvm_object base class, is also an empty method, so if field macros are used to define the fields of the transaction, the built-in copy() method will be populated with the proper code to copy the transaction fields from the field macro definitions and then it will execute the empty do_copy() method, which will perform no additional activity.

The **copy()** method can be used as needed in the UVM testbench. One common place where the copy() method is used is to copy the sampled transaction and pass it into a sb_calc_exp() (scoreboard calculate expected) external function that is frequently used by the scoreboard predictor.

The **clone()** method calls the create() method (constructs an object of the same type) and then calls the copy() method. It is a one-step command to create and copy an existing object to a new object handle.

21. What is the UVM factory?

Ans: UCM Factory is used to manufacture (create) UVM objects and components. Apart from creating the UVM objects and components the factory concept essentially means that you can modify or substitute the nature of the components created by the factory without making changes to the testbench.

For example, if you have written two driver classes, and the environment uses only one of them. By registering both the drivers with the factory, you can ask the factory to substitute the existing driver in environment with the other type. The code needed to achieve this is minimal, and can be written in the test.

22. What are the types of sequencer? Explain each?

Ans: There are two types of sequencers :

**uvm_sequencer #(REQ, RSP) :**

When the driver initiates new requests for sequences, the sequencer selects a sequence from a list of available sequences to produce and deliver the next item to execute. In order to do this, this type of sequencer is usually connected to a driver uvm_driver #(REQ, RSP).

**uvm_push_sequencer #(REQ, RSP) :**
The sequencer pushes new sequence items to the driver, but the driver has the ability to block the item flow when its not ready to accept any new transactions. This type of sequencer is connected to a driver of type uvm_push_driver #(REQ, RSP).

23. What are the different phases of uvm_component? Explain each?
Ans:

24. How set_config_* works?
Ans: The *uvm_config_db* class provides a convenience interface on top of the uvm_resource_db to simplify the basic interface that is used for configuring uvm_component instances.

Configuration is a mechanism in UVM that higher level components in a hierarchy can configure the lower level components variables. Using ***set_config_**** methods, user can configure integer, string and objects of lower level components. Without this mechanism, user should access the lower level component using hierarchy paths, which restricts re-usability.

This mechanism can be used only with components. Sequences and transactions cannot be configured using this mechanism. When ***set_config_**** method is called, the data is stored w.r.t strings in a table. There is also a global configuration table.

Higher level component can set the configuration data in level component table. It is the responsibility of the lower level component to get the data from the component table and update the appropriate table.

following are the method to configure integer, string and object of uvm_object based class
function void **set_config_string** (string inst_name, string field_name, string value)

function void **set_config_object** (string inst_name, string field_name, uvm_object value, bit clone =1)

25. What are the advantages of the uvm RAL model?
Ans: The RAL (register abstraction layer) provides accesses to DUT and also keeps a track of register content of DUT.
UVM RAL can be used to automate the creation of high level, object oriented abstraction model of registers and memory in DUT.
Register layer makes the register abstraction and access of its contents independent of the bus protocol which is used to transfer data in and out of registers inside the design.
Hierarchical model provided by RAL makes the reusability of test bench components very easy.
The changes in initial configuration of registers or specifications can be easily communicated in the entire environment.
RAL layer supports both front door and backdoor access. The backdoor access does not use the bus interface rather it uses the HDL defined paths for direct communication with the device. Thus in zero simulation time the registers of device can be reconfigured using the backdoor access and verification can be started.
One more advantage of backdoor access is that it can be used for verify if the access through front door are happening correctly. To achieve this the front door, write is verified using backdoor read.

26. What is the different between set_config_* and uvm_config_db?
Ans:

27. What are the different override types?
Ans: Two type of overriding is supported by UVM

**1. Type overriding**
Type overriding means that every time a component class type is created in the Testbench hierarchy, a substitute type i.e. derived class of the original component class, is created in its place. It applies to all the instances of that component type.

Syntax:
<original_type>::type_id::set_type_override(<substitute_type>::get_type(), replace);

where “replace” is a bit which is when set equals to 1, enables the overriding of an existing override else existing override is honoured.

**2. Instance overriding**

In Instance Overriding, as name indicates it substitutes ONLY a particular instance of the component OR a set of instances with the intended component. The instance to be substituted is specified using the UVM component hierarchy.

Syntax:
<original_type>::type_id::set_inst_override(<substitute_type>::get_type(), <path_string>);

Where “path_string” is the hierarchical path of the component instance to be replaced.

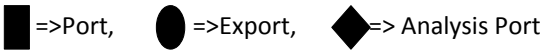
28. What is virtual sequence and virtual sequencer?
Ans: A virtual sequencer is a sequencer that is not connected to a driver itself, but contains handles for sequencers in the testbench hierarchy. It is an optional component for running of virtual sequences - optional because they need no driver hookup, instead calling other sequences which run on real sequencers.

A sequence which controls stimulus generation across more than one sequencer, coordinate the stimulus across different interfaces and the interactions between them. Usually the top level of the sequence hierarchy i.e. 'master sequence' or 'coordinator sequence'. Virtual sequences do not need their own sequencer, as they do not link directly to drivers. When they have one it is called a virtual sequencer.

29. Explain the end of the simulation in UVM?
Ans: Different approaches to finish the UVM Test using the objection mechanism are
1. Raising & dropping objections
raise_objection() and drop_objection() are the methods to be used to do that.
2. phase_ready_to_end
phase_ready_to_end method is executed automatically by UVM once ‘all dropped’ condition is achieved during Run Phase.
3. set_drain_time
Another approach supported by UVM is setting the drain time for the simulation environment. Drain time concept is related to the extra time allocated to the UVM environment to process the left over activities e.g. last packet analysis & comparison etc after all the stimulus is applied & processed.

30. How to declare multiple imports?
Ans: multiple imports are declared using analysis port
**uvm_analysis_imp**
Receives all transactions broadcasted by a uvm_analysis_port.

31. What is the symbolic representation of port, export and analysis port?
Ans:




32. What is the difference in usage of $finish and global stop request in UVM?
Ans: The verilog $finish task does actually print the line number and file name along with time where it stopped.
However, global_stop_request() does not.

33. Why we need to register class with the uvm factory?
Ans: UVM factory is a mechanism to improve flexibility and scalability of the testbench by allowing the user to substitute an existing class object by any of its inherited child class objects.

34. can we use set_config and get_config in sequence?
Ans: set_config_* can be used only for the components not for the sequences.
By using configuration you can change the variables inside components only not in sequences.
So from the sequence, using the sequencer get_config_* methods, sequence members can be updated if the variable is configured.

35. What is the uvm_heartbeat ?
Ans: Heartbeats provide a way for environments to easily ensure that their descendants are alive. A uvm_heartbeat is associated with a specific objection object. A component that is being tracked by the heartbeat object must raise (or drop) the synchronizing objection during the heartbeat window.

II. System Verilog Questions

1. What is the difference between an initial and final block of the system verilog?
Ans. Initial block is getting executed at start of simulation while Final block is getting executed at end of simulation.
Both of them gets executed only once during the simulation, You can schedule an event or have delay in initial block But you can’t schedule an event or have delay in final block.

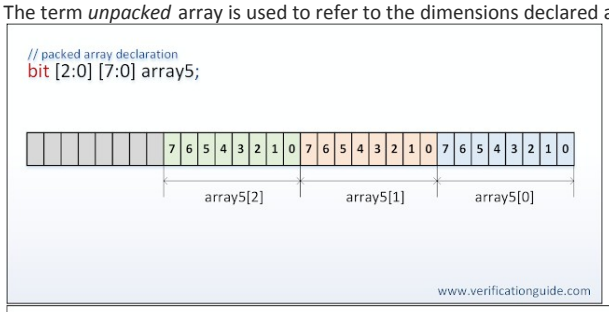
2. Explain the simulation phases of **System Verilog verification**?

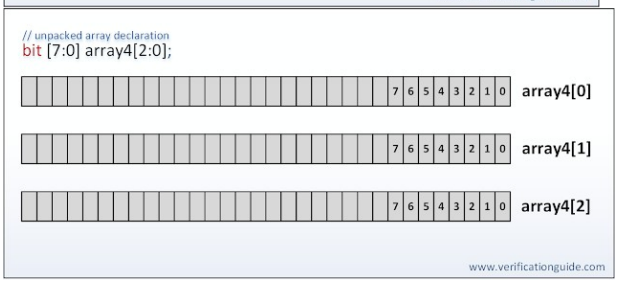
Ans. Build phase, connect phase and Run phase

3. What is the Difference between SystemVerilog packed and unpacked array?

Ans.

- The term *packed* array is used to refer to the dimensions declared before the data identifier name
- The term *unpacked* array is used to refer to the dimensions declared after the data identifier name




4. What is “This ” keyword in the system verilog?

Ans: The this keyword is used to **refer to class properties, parameters and methods of the current instance**.

5. What is alias in SystemVerilog?

Ans: The **alias** construct has largely been replaced by packed structures and the **let** construct. It is a way of providing a more user friendly name for another signal, or a select of another signal.

6. randomized in the systemverilog test bench?

Ans: Randomization is the process of making something random; SystemVerilog randomization is the **process of generating random values to a variable**.

7. in SystemVerilog which array type is preferred for memory declaration and why?

Ans: Associative arrays are better to model large arrays as memory is allocated only when an entry is written into the array. Dynamic arrays on the other hand need memory to be allocated and initialized before using.(When the size of the collection is unknown or the data space is sparse, an associative array is a better option. In associative array, it uses the transaction names as the keys in associative array.

e.g. int array[string];)

8. How to avoid race round condition between DUT and test bench in System Verilog verification?

Ans. In test bench, if driving is done at posedge and reading in DUT is done at the same time , then there is race.

9. What are the advantages of the systemverilog program block?

Ans: To create a container to hold all other testbench data such as tasks, class objects and functions

- Avoid race conditions with the design by getting executed during the reactive region of a simulation cycle

10. What is the difference between logic and bit in SystemVerilog?

Ans: As we know "**logic**" data type has 4 states = 0, 1, X & Z, where as "**bit**" has only 2 states = 0 & 1.

11. What is the difference between datatype logic and wire?

Ans: Wire is verilog datatype whereas logic is SystemVerilog data type.

12. What is a virtual interface?

Ans: A virtual interface is a variable that represents an interface instance.

- Virtual interfaces can be declared as class properties, which can be initialized procedural or by an argument to new()

13. What is an abstract class?

Ans: SystemVerilog class declared with the keyword virtual is referred to as an *abstract* class.

14. What is the difference between $random and $urandom?

Ans: The system function $urandom provides a mechanism for generating pseudorandom numbers. The function returns a new 32-bit random number each time it is called. The number shall be unsigned.

$random() is same as $urandom() but it generates signed numbers.

15. What is the expect statements in assertions?

Ans: An expect statement is very similar to an assert statement, but it must occur within a procedural block (including initial or always blocks, tasks and functions), and is used to block the execution until the property succeeds.

16. What is DPI?

Ans: SystemVerilog DPI (**Direct Programming Interface**) is an interface which can be used to interface SystemVerilog with foreign languages.

17. What is the difference between == and === ?

Ans: == For comparing bits (0 or 1) === For comparing all 4 states (0, 1, x, z).

18. What are the system tasks?

Ans: Tasks and Functions provide a means of splitting code into small parts.

A Task can contain a declaration of parameters, input arguments, output arguments, in-out arguments, registers, events, and zero or more behavioral statements.

19. What are parameterized classes?

Ans: Parameterized classes are same as the parameterized modules in the verilog. parameters are like constants local to that particular class. The parameter value can be used to define a set of attributes in class. default values can be overridden by passing a new set of parameters during instantiation. this is called parameter overriding.

20. How to generate array without randomization?

Ans: Implement your own pseudo-random number generator algorithm, such as a simple linear feedback shift register.
Use an older Verilog randomization function, such as $urandom.

21. What is the difference between always_comb() and always@(*)?

Ans: always_comb is sensitive to changes within the contents of a function, whereas always @* is only sensitive to changes to the arguments of a function.
i.e., whenever the variables inside a function will change, the always_comb will trigger, but in your case, always @(*) will only trigger if we pass an argument in the function.
So either you can use,

```
always @(*) begin
c = my_func(b);
end
```

or you can use,

```
always_comb begin
c = my_func();
end
```

22. What is the difference between overriding and overloading?

Ans: **When two or more methods in the same class have the same name but different parameters**, it's called Overloading. When the method signature (name and parameters) are the same in the superclass and the child class, it's called Overriding.

23. Explain the difference between **deep copy** and **shallow copy**?

Ans: Shallow Copy reflects changes made to the new/copied object in the original object. Deep copy doesn't reflect changes made to the new/copied object in the original object.

24. What is interface and advantages over the normal way?

Ans: Interface allows the number of signals to be grouped together and represented as a single port, the single port handle is passed instead of multiple signal/ports.
interface declaration is made once and the handle is passed across the modules/components.
addition and deletion of signals are easy.

25. What is modport and explain the usage of it?

Ans: The Modport groups and specifies the port directions to the wires/signals declared within the interface. modports are declared inside the interface with the keyword modport.

26. What is a clocking block?

Ans: A clocking block specifies timing and synchronization for a group of signals.

- The set of signals that will be sampled and driven by the testbench

27. What is the difference between the clocking block and modport?

Ans: A clocking block specifies timing and synchronization for a group of signals.
Modports defines inputs and outputs

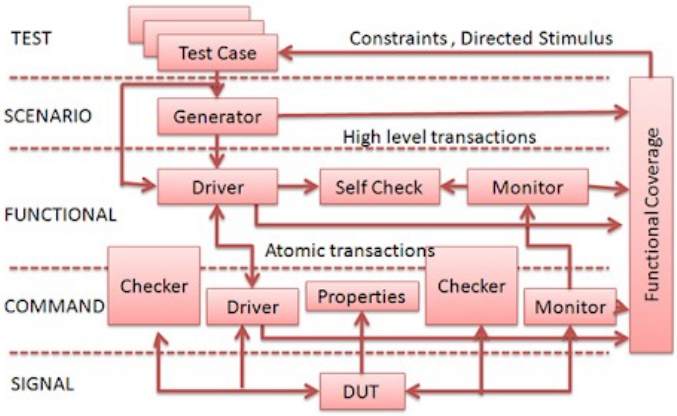
28. What are the different types of verification approaches?

Ans: **These approaches works in tandem & together makes a best possible solution.**

- Directed Verification.
- Constrained Random Verification.
- Coverage Driven Verification.
- Assertion Based Verification.
- Emulation Based Verification.

29. What are the different layers of layered architecture?

Ans: The layered TestBench is the heart of the verification environment in VMM:
i. signal layer:
This layer connects the TestBench to the RTL design. It consists of interface, clocking, and modport constructs.
ii. command layer:
This layer contains lower-level driver and monitor components, as well as the assertions. This layer provides a transaction-level interface to the layer above and drives the physical pins via the signal layer.
iii. functional layer:
This layer contains higher-level driver and monitor components, as well as the self-checking structure (scoreboard/tracker).
iv. scenario layer:
This layer uses generators to produce streams or sequences of transactions that are applied to the functional layer. The generators have a set of weights, constraints or scenarios specified by the test layer. The randomness of constrained-random testing is introduced within this layer.
v. test layer:
Tests are located in this layer. Test layer can interact with all the layers. This layer allows to pass directed commands to functional and command layer.






30. What is the difference between a $rose and @ (posedge)?
Ans: the system task $rose is used to detect a +ve edge of the given signal.
posedge returns event where as $rose returns a boolean value. Events cannot be used in expression, $rose can be used.

31. What is the use of extern?
Ans: the extern keyword is used **to extend the visibility of variables/functions**. Since functions are visible throughout the program by default, the use of extern is not needed in function declarations or definitions. Its use is implicit. When extern is used with a variable, it's only declared, not defined.

32. What is scope randomization?
Ans: std::randomize(), also called Scope-Randomize Function, is a utility provided by the SystemVerilog standard library (that's where the std:: comes from). It gives you the ability to randomize variables that are not members of a Class.

33. What is the difference between blocking and non-blocking assignments?
Ans: "blocking" and "nonblocking" assignments **only exist within always blocks**. A blocking assignment takes affect immediately it is processed. A nonblocking assignment takes place at the end of processing the current "time delta"

34. what are automatic variables?
Ans: An automatic variable exists **only for the lifetime of the task, function or block** - they are created when the task, function or block is entered and destroyed when it is left.

35. What is the scope of local and private variables?
Ans: the scope of local variable is, from declaration to end of containing block, the scope of private variable is Any method in class public: Package that contains class, packages that have access to this package

36. How to check if any bit of the expression is X or Z?
Ans: assert property( @(posedge clk) $isunknown(bus) ) ;

- $isunknown(expression): checks if any bit of the expression is X or Z.

37. What is the Difference between param and typedef?
Ans: in case of using typedef enum multiple variables are declared of the same type. Since in this case, parameters it will work just as well.

38. what is timescale?
Ans: Timescale **specifies the time unit and time precision of a module that follow it**. The simulation time and delay values are measured using time unit.

39. Explain the difference between new( ) and new[ ] ?
Ans: Both new() and new[] allocate memory and initialize values. The big difference is that the new() function is called to construct a single object, whereas the new[] operator is building an array with multiple elements.

40. What is the difference between task and function in class and Module?
Ans: A function is meant to do some processing on the input and return a single value, whereas a task is more general and can calculate multiple result values and return them using output and inout type arguments. Tasks can contain simulation time consuming elements such as @, posedge and others.

41. Why always blocks are not allowed in the program block?
Ans: Because an always block never terminates, it was kept out of the program block so the concept of test termination would still be there.

42. Why forever is used instead of always in program block?
Ans: The difference between forever and always is that always can exist as a "module item", which is the name that the Verilog spec gives to constructs that may be written directly within a module, not contained within some other construct. initial is also a module item. always blocks are repeated, whereas initial blocks are run once at the start of the simulation.

43. What is SVA?
**Ans: SystemVerilog Assertions** (SVA) is essentially a language construct which provides a powerful alternate way to write constraints, checkers and cover points for your design.

44. Explain the difference between fork-join, fork-join_none, and fork- join_any?
Ans:

45. What is the difference between mailboxes and queues?
Ans: A simple queue can only push and pop items from either the front or the back. However, a mailbox is a built-in class that uses semaphores to have atomic control the push and pop from the queue.

46. What is casting?
Ans: casting means the conversion of one data type to another datatype.

47. What is inheritance and polymorphism?
Ans: Inheritance enables reuse. It's called *inheritance* because all the existing properties and methods of an original base (or *super* ) class are passed on to the newly created class, called an extended (or *derived* ) class.

48. What is callback?
Ans: SystemVerilog callback specifies the *rules* to **define the methods** and **placing method calls** to achieve '**a return call** to methods'.

49. What is constraint solve-before?
Ans: solve before constraints are **used to force the constraint solver to choose the order in which constraints are solved**. constraint solver will give equal weight-age to all the possible values. i.e On multiple randomization solver should assign all the possible values.

50. What is coverage and what are different types?
Ans: Coverage is used to measure tested and untested portions of the design. Coverage is defined as the percentage of verification objectives that have been met. There are two types of coverage metrics, **Code Coverage**. **Functional Coverage**.

51. What is the importance of coverage in SystemVerilog verification?
Ans: An important consideration in coverage sampling is **to ensure that data is relevant** (e.g., not sampling DUT outputs during the reset or configuration phase).

52. When you will say that verification is completed?
Ans: Your verification is done, **if you met all your goals defined in your verification plan**.

53. What are illegal bins? Is it good to use it and why?
Ans: Hitting a illegal bin **can cause simulator to terminate simulation**. Normally illegal bin syntax should be used on coverage points on variables inside DUT or on ports which are output of DUT. Having illegal bin syntax on testbench stimulus could prevent error injection.

54. What is the advantage of seed in randomization?
Ans: It **increases the probability of a different result**. It also changes the distribution of results when you ask for sequences of random results. Note that each seed does produce a unique sequence of 4 numbers

55. What is circular dependency?
Ans: A circular dependency is where Project A depends on something in Project B and project B depends on something in Project A. This means to compile Project A you must first compile Project B, but you can't do that as B requires A to be compiled. This is the problem that circular dependencies cause.

56. What is "super "?
Ans: T he super keyword is **used from within a sub-class to refer to properties and methods of the base class**. It is mandatory to use the super keyword to access properties and methods if they have been overridden by the sub-class.

57. What is the input skew and output skew in the clocking block?
Ans: If an input skew is mentioned for a clocking block, then **all input signals within that block will be sampled at skew time units before the clock event**. If an output skew is mentioned for a clocking block, then all output signals in that block will be driven skew time units after the corresponding clock event.

58. What is a static variable?
Ans: **Static variables get allocated and initialized before time 0** and are never deallocated.

59. What is a package?
Ans: Packages **provide a mechanism for storing and sharing data, methods, property, parameters that can be re-used in multiple other modules, interfaces or programs**.

**60. What is the difference between bit [7:0] and byte?**
**Ans: byte is signed whereas bit** [7:0] is unsigned.

61. What is randomization and what can be
Ans: Randomization is the process of making something random; SystemVerilog randomization is **the process of generating random values to a variable**. Verilog has a $random method for generating the random integer values.

62. What are the constraints? Is all constraints are bidirectional?
Ans: SystemVerilog provides this control using constraints. A constraint is **a Boolean expression describing some property of a field**. Constraints direct the random generator to choose values that satisfy the properties you specify in your constraints.

63. What are in line constraints?
Ans: An inline constraint is a **constraint you declare on the same line as the column when creating a table**.

64. What is the difference between rand and randc?
Ans:

65. Explain pass by value and pass by ref?
Ans: In SystemVerilog, Pass by value is **the default mechanism for passing arguments to subroutines**. This argument passing mechanism works by copying each argument into the subroutine area. If the subroutine is automatic, then the subroutine retains a local copy of the arguments in its stack.

66. What are the advantages of cross-coverage?
Ans: Cross **allows keeping track of information which is received simultaneous on more than one cover point**.

67. What is the difference between associative and dynamic array?
Ans: A dynamic array gets created with a **variable size** and stays that size in a contiguous block of memory. Its elements are indexed starting with integer 0. The benefit of an associative array is since each element gets allocated individually, you don't need to allocate a contiguous set of array elements.

68. What is the type of SystemVerilog assertions?
Ans: In SystemVerilog there are two kinds of assertions: **immediate (assert) and concurrent (assert property)**. Coverage statements (cover property) are concurrent and have the same syntax as concurrent assertions, as do assume property statements.

69. What is the difference between $display,$strobe,$ monitor?
Ans: The only difference between the two is that **$display writes out a newline character at the end of the text**, whereas $write does not. $strobe prints the text when all simulation events in the current time step have executed.

70. Can we write SystemVerilog assertions in class?
Ans: Assertions can also access static variables defined in classes; however, access to dynamic or rand variables is illegal. Concurrent assertions are illegal within classes, but can **only be written in modules, SystemVerilog interfaces**, and SystemVerilog checkers2.

71. What is an argument pass by value and pass by reference?
Ans: By definition, pass by value means **you are making a copy in memory of the actual parameter's value that is passed in**, a copy of the contents of the actual parameter. ... In pass by reference (also called pass by address), a copy of the address of the actual parameter is stored.